



Optimizing convolutional neural networks architecture using a modified particle swarm optimization for image classification

D. Elhani^a, A.C. Megherbi^a, A. Zitouni^a, F. Dornaika^{b,*}, S. Sbaa^a, A. Taleb-Ahmed^{c,**}

^a University of Biskra, Department of Electrical Engineering, Biskra, Algeria

^b Ho Chi Minh City Open University, 97 Vo Van Tan, Ward Vo Thi Sau, District 3, 70000 Ho Chi Minh City, Vietnam

^c Univ. Polytechnique Hauts-de-France, Univ. Lille, CNRS, Centrale Lille, UMR CNRS 8520, F-59313 Valenciennes, France

ARTICLE INFO

Keywords:

Convolutional neural networks
Neural network design
Hyperparameters
Particle swarm optimization
Image classification

ABSTRACT

Although Convolutional Neural Networks (CNNs) have been shown to be highly effective in image classification tasks, designing their architecture to achieve optimal results is often challenging. This process is time consuming, requires significant effort and expertise, and is complicated by the large number of hyperparameters. To address this problem, in this work we propose an approach that reduces human intervention and automatically generates the best CNN design. Our approach uses a variant of Particle Swarm Optimization (PSO), called Particle Swarm Optimization without Velocity (PSWV), to speed up convergence and reduce the number of iterations required to determine the optimal CNN hyperparameters. We developed a novel strategy to determine the updated position of each particle using a linear combination of the best position of the particle and the best position of the swarm without relying on the velocity equation. Our algorithm harnesses the power of the variable-length encoding strategy to represent particles within the population, thereby providing swift convergence towards the best architecture. We evaluate our proposed algorithm against several recent algorithms in the literature by using nine benchmark datasets for classification tasks and comparing it to 27 other algorithms, including state-of-the-art ones. Our experimental results show that our proposed method, pswvCNN, is able to quickly find effective CNN architectures that provide comparable performance to the best currently available designs, indicating its significant potential.

1. Introduction

The Convolutional Neural Network (CNN) is a deep learning architecture that was inspired by the visual system's structure, as observed by Hubel and Wiesel in 1962 (Hubel & Wiesel, 1962). CNN is one of the most effective learning algorithms for understanding image content and has exhibited exceptional performance in image segmentation, classification, and detection, as noted by (Liu, Deng, & Yang, 2019). CNN operates directly on raw data without needing to extract image features, as these are embedded within the CNN architecture.

Human experts have developed a wide range of CNN models, including highly successful ones like VGG16 (Simonyan & Zisserman, 2014), Inception (Szegedy et al., 2015), ResNet (He, Zhang, Ren, and Sun, 2016), DenseNet (Huang, Liu, Van Der Maaten, & Weinberger, 2017) and SENet (Hu, Shen, & Sun, 2018). These models were meticulously crafted by taking into account the specific characteristics and requirements of their respective problem domains.

The success of VGG16 lies in its ability to demonstrate that neural networks with a greater number of layers and reduced convolutional filter size can surpass the performance of networks with fewer layers. Inception proved the feasibility of using the concept of network-in-network to develop computationally efficient deep neural networks. ResNets incorporate shortcut connections, also known as skip or residual connections, to link a layer's inputs to its outputs. ResNet employ residual mapping instead of direct mapping between inputs and outputs, leading to a simplified training process. DenseNets link the outputs of each layer to every subsequent layer, similar to the connectivity pattern of fully-connected neural networks.

The basic building blocks of a CNN architecture are the convolutional layer, the pooling layer, and the fully-connected concatenated layer (Fc). It performs the convolution operation by sliding filters over the input image and applying an activation function that results in feature maps. This is followed by the pooling layer, which compresses the information by reducing the size of the intermediate image. This

* Corresponding author at: Ho Chi Minh City Open University, 97 Vo Van Tan, Ward Vo Thi Sau, District 3, 70000 Ho Chi Minh City, Vietnam.

** Corresponding author.

E-mail addresses: djenaihi.elhani@univ-biskra.dz (D. Elhani), ac.megherbi@univ-biskra.dz (A.C. Megherbi), athmane.zitouni@univ-biskra.dz (A. Zitouni), fdornaika@gmail.com (F. Dornaika), s.sbaa@univ-biskra.dz (S. Sbaa), Abdelmalik.Taleb-Ahmed@uphf.fr (A. Taleb-Ahmed).

<https://doi.org/10.1016/j.eswa.2023.120411>

Received 20 June 2022; Received in revised form 22 March 2023; Accepted 5 May 2023

Available online 12 May 2023

0957-4174/© 2023 Elsevier Ltd. All rights reserved.

process is repeated several times, and in the end, there are one or more fully-connected layers in the architecture. The last Fc layer represents the classifier. CNN uses two different sets of parameters: trainable parameters and hyperparameters. The trainable parameters are learned during the learning phase, but the values of the hyperparameters must be set before learning begins. One should find the optimal combination of hyperparameter values in a reasonable time. This is a difficult task because it is very computationally intensive.

In Deep Learning, the model's performance depends heavily on the hyperparameter values chosen. Hyperparameter optimization aims to search through different hyperparameter configurations to find one that performs the best. Usually, hyperparameter optimization is done in tedious manual work since the search space is enormous, and the evaluation of each configuration can be expensive.

In practice, it is not easy to evaluate the network hyperparameters a priori. Indeed, the number of layers, the number of neurons per layer, or the different connections between layers are crucial elements and are essentially determined by good intuition or by a series of test/error calculations, which is a time-consuming process (Bacanin, Bezdan, Tuba, Strumberger, & Tuba, 2020b).

There are many approaches to hyperparameter optimization. The popular methods include grid search, random search (Bergstra & Bengio, 2012), Bayesian optimization (Martinez-Cantin, 2014), and metaheuristic algorithms. Grid Search is the most common method for searching the hyperparameter configuration space to find the optimal configuration by evaluating all possibilities in the search space; in random search, we define a search space as a bounded domain of hyperparameter values and randomly sample points in that domain.

Since metaheuristic algorithms have been used to solve high-dimensional and nonconvex optimization problems, many recent studies have used metaheuristic algorithms to solve optimization problems with CNN hyperparameters. Metaheuristics (modern optimization techniques) have been successfully used to solve optimization problems in the field of science and engineering as well as in industry. Some researchers have done a good work in automatically evolving CNN architectures by using: Genetic Algorithm (Aszemi & Dominic, 2019; Loussaief & Abdelkrim, 2018; Sun, Xue, Zhang, & Yen, 2019; Sun, Xue, Zhang, Yen, & Lv, 2020), Genetic Programming (Suganuma, Shirakawa, & Nagao, 2017), PSO (Fregoso, Gonzalez, & Martinez, 2021; Junior & Yen, 2019; Lawrence, Zhang, Lim, & Phillips, 2021; Li, Xiao, Chen, & Jiao, 2019; Nistor & Czibula, 2022; Singh, Chaudhury, & Panigrahi, 2021; Singh et al., 2021; Sun et al., 2019; Wang, Sun, Xue, & Zhang, 2018), Symbiotic Organism Search Algorithm (SOS) (Miao, Yao, & Zhao, 2021), Monarch Butterfly Optimization (MBO) (Bacanin, Bezdan, Tuba, Strumberger, & Tuba, 2020a), and Firefly Algorithm (Strumberger et al., 2019). In this work, CNN architectures are automatically designed using metaheuristic algorithms, and all of these techniques are evaluated on well-known benchmark datasets. IPPSO (Wang et al., 2018) stands as one of the primary approaches that applied PSO to tackle the problem of CNN architecture. The IPPSO model incorporates adaptive coding inspired by IP addressing and subnet research in computer networks. In particular, an IP addressing policy represents layer parameters as a series of bits (0,1). The IPPSO algorithm was applied to a population of 20 particles, where the maximum length of particle layers is 9, the maximum number of fully-connected layers is 3, and the number of training epochs before evaluating the trained CNN is 10.

psocNN (Junior & Yen, 2019) is another state-of-the-art PSO-based architecture generation method. It uses a test methodology consisting of 20 particles, a maximum particle layer length between 3 and 20, a kernel size between 3×3 and 7×7 , and a maximum of 256 channels. The model uses a traditional PSO algorithm where the position of a particle is changed by randomly copying layers from the best personal or global solutions.

Li et al. employed an evolution technique and developed an optimization strategy to evolve the CNN architecture using quantum-behaved particle swarm optimization (BQPSO) in Li et al. (2019). In

BQPSO, where there are no concepts of velocities and trajectories, only position and distance, each particle in the population (CNN architecture) is encoded by a fixed-length binary string for its layers. The convolutional layer is encoded by a binary string consisting of an ID followed by its parameter. Similarly, all other elements such as max-pooling and Fc (Fully-connected) are encoded, and finally, all binary strings are concatenated to represent a particle for a CNN. A randomly generated fixed-length binary string does the initialization of the population. Fitness is calculated by running the CNN model with the selected parameters and then evolving the solution using PSO.

Lawrence et al. (2021) propose a novel approach to automatically evolve convolutional neural networks for image classification using particle swarm optimization (PSO). The approach uses a group-based coding strategy that includes at least one convolutional layer and a pooling layer as the last layer in each group. The authors also introduce a new mechanism for updating velocity and position, which uses weighted variables to explore the search space and generate diverse solutions. The proposed approach is evaluated on popular image classification datasets, and the experimental results show the effectiveness and competitiveness of the proposed method compared to state-of-the-art approaches. The article provides a valuable contribution to the field of automated deep learning and demonstrates the potential of PSO for evolving CNNs for image classification.

In Lorenzo, Nalepa, Ramos, and Pastor (2017), a parallel version of the PSO algorithm is proposed for hyperparameter optimization in DNNs. The particle array (each representing a combination of hyperparameter values) is evolved in the parallel search for hyperparameters that give the best possible performance for the classification of the base elements of the DNN model.

sosCNN [Miao et al., 2021] is also selected for comparison. The sosCNN method was proposed by Miao et al. (2021) for deep network generation. It employed a symbiotic organism search (SOS) algorithm for the search of evolving network architectures, by introducing two new strategies. First, a slack gain strategy was proposed for devising architectures with greater depths because the original SOS algorithm excessively eliminated deep networks early in the search process. Second, a dissimilar mutation strategy for enhancing the global exploration ability. According to the authors, this procedure aided in ensuring faster convergence. However, both sosCNN and psocNN do not construct models comprising residual connections; therefore, they are susceptible to the vanishing gradient problem. A swarm of 20 individuals is employed in this method and optimized through 10 iterations. The hyperparameters chosen for optimization consist of layer types, including convolution layers with kernel sizes ranging from 3×3 to 7×7 , up to 256 filters, pooling layers, or fully-connected layers. The total number of layers is restricted to a maximum of 20.

The MPSO-CNN algorithm was introduced by Singh et al. (2021) as a novel approach to simultaneously determining the architecture and hyperparameters of a Convolutional Neural Network (CNN) using a Multi-level Particle Swarm Optimization (MPSO) algorithm. The method involves employing multiple swarms at two different levels, where the initial swarm at level 1 is responsible for optimizing the architecture, and multiple swarms at level 2 optimize the hyperparameters. To enhance the exploration and exploitation capabilities of the particles, a sigmoid-like inertia weight was utilized. The proposed solution showcases promising results and represents an innovative approach to CNN optimization.

IntelliSwAS (Nistor & Czibula, 2022) is a novel approach for optimizing deep neural network (DNN) architectures using a PSO algorithm. The approach works by initializing a swarm of particles, where each particle represents a potential DNN architecture. Each particle is then evaluated based on a fitness function that measures the accuracy of the corresponding DNN model. The fitness value is used to update the position of the particle in the search space. During the PSO-based optimization process, the particles in the swarm communicate with each other and adjust their positions based on their own experience

and the experience of the swarm as a whole. IntelliSwASincorporates several novel features to improve its performance. For instance, it uses an intelligent strategy for initializing the swarm, which uses prior knowledge about DNN architectures to guide the search process. It also introduces a novel method for updating the particles, which incorporates information about the best-performing architectures found during the search process.

Thus, the PSO algorithms have many advantages. These algorithms are intuitive, easy to program, and have a relatively small number of parameters compared to other population metaheuristics. They allow the optimization of arbitrary objective functions (numerical, symbolic functions, and algorithms) of arbitrary dimensions and use heterogeneous variables.

The particle swarm optimization (PSO) algorithm has the disadvantage of quickly falling into a local optimum, having a low convergence rate in the iterative process, and losing the diversity of populations (Li, Du, & Nian, 2014).

An adaptive particle swarm without a PSWV velocity equation is proposed to solve these problems. In this proposed variant, not only the diversity of the algorithm was improved, but also the fall of particles into the local optimum was avoided. The simulation results show that this variant can effectively avoid the problem of premature convergence. Compared to other algorithms, the convergence rate is faster (El-Sherbiny, 2011).

This research aims to find a CNN structure for image classification that compromises the search speed for the best hyperparameters and the classification accuracy. Therefore, we present an approach that uses a novel PSO without velocity equation PSWV with a new particle update strategy to solve such an optimization problem. We call this approach pswvCNN. pswvCNN focuses on the optimization of CNN hyperparameters that determine the architecture of the CNN and thus affect the classification accuracy. The paper offers the following major contributions:

1. A novel variant of the PSO algorithm called the particle swarm-without-velocity optimization algorithm can be used to find the best architectures for convolutional neural networks as the lengths of the population particles are different and variable. This algorithm can significantly reduce the number of iterations required to reach reasonable solutions.
2. A novel update strategy of each particle is determined directly from the combination between its own best position and the best position of the population, without the need for the velocity equation.
3. A fast optimization algorithm finds the best solutions to the CNN architectural design optimization problem.

The rest of the paper is organized as follows. In Section 2, we give an overview of CNN, the PSO and PSWV algorithms, and related techniques that address the problem of finding the best architecture for CNN. Section 3 gives a detailed insight into our proposed method. Section 4 describes our experiments and reports the results. Section 5 provides a detailed analysis of the experimental results. Finally, Section 6 contains our conclusion, limitations and discusses possible future directions.

2. Background and related work

2.1. Convolutional neural network

In this part, we will focus on one of the most powerful algorithms in Deep Learning, the Convolutional Neural Network. The Convolutional Neural Network (CNN or ConvNet) is an artificial neural network used in image recognition and processing, specifically designed to process pixel data. The visual cortex of animals inspires CNN.

Fig. 1 demonstrates a single CNN's global design, which includes two convolutional processes, two pooling operations, four sets of feature maps, and a fully-connected layer at the model's tail.

The first part of a CNN is the actual convolution part. This is a simple mathematical operation typically used for image processing and recognition to extract features from images. An image is passed through a series of filters or convolution kernels, creating new images called feature maps.

In addition to filtering, the advantage of the convolution kernel of a CNN is that it allows the extraction of features specific to each image by compressing the images to reduce their original size (pooling method). Two standard pooling methods are average pooling and maximum pooling, which summarize the average presence and the most activated presence of a feature, respectively.

The Rectified Linear Units (ReLU) activation layer: this layer replaces all negative values received as inputs with zeros. The interest of these activation layers is to make the model nonlinear and, therefore, more complex.

Fully-connected layer (Fc): these layers are located at the end of the CNN architecture and are fully-connected to all output neurons. After receiving an input vector, the Fc layer sequentially applies a linear combination and then an activation function with the final goal of classifying the image input. Finally, it returns a vector of size d corresponding to the number of classes, where each component represents the probability that the image input belongs to a class.

Hyperparameters are the variables that determine the network architecture (e.g., Number of channels, number of layers, kernel/filter size, padding, stride, pooling layer parameters, ...) and the variables that determine how the network is trained (e.g., Learning rate, dropout rate, normalization, regularization (L_1 , L_2), activation function type (ReLU, Sigmoid, Tanh), batch size, as shown in Table 1.

The hyperparameters are defined before training (before optimizing weights and biases).

2.2. Particle swarm optimization

The PSO algorithm, which utilizes a population of potential solutions to converge towards an optimal solution, is an evolutionary algorithm introduced by R. Eberhart and J. Kennedy in 1995 (Kennedy & Eberhart, 1995). Since then, PSO has been considered an important metaheuristic optimization tool used in many areas of science (Hu, Zhang, Li, & Deng, 2021; Mosa, 2020).

It is inspired by the collective movements observed in certain social animals such as fish and migratory birds, which tend to mimic the successful behaviors they observe in their environment (Sibaliya, 2019). In PSO, the particle i is modeled by its position vector $x_i = (x_{i1}, x_{i2}, \dots, x_{iD})$ and by its velocity vector $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$. The quality of its position is determined by the value of the objective function at that point. This particle stores the best position it has ever traversed, which we denote by $pBest_i = (pBest_{i1}, pBest_{i2}, \dots, pBest_{iD})$. The best position reached by the particles of the swarm is denoted by $gBest = (gBest_1, gBest_2, \dots, gBest_D)$. We refer to the global version of PSO, where all particles in the swarm are considered as neighbors of particle i , hence the notation $gBest$ (global best).

PSO can be performed in the following steps:

The first step is to initialize the positions, velocities, and best position ($pBest_{ij}$) of the particles.

In the second step, calculate the global best position by evaluating the fitness values for each particle based on its initial positions.

Third, update the particles' best previous position ($pBest_{ij}$) and best global position ($gBest_j$).

Fourth: In iteration $t + 1$, calculate the velocity vector and the position vector from Eq. (1) and Eq. (2), respectively.

Fifth, if the termination condition is satisfied, the process is complete; otherwise, return to step (three).

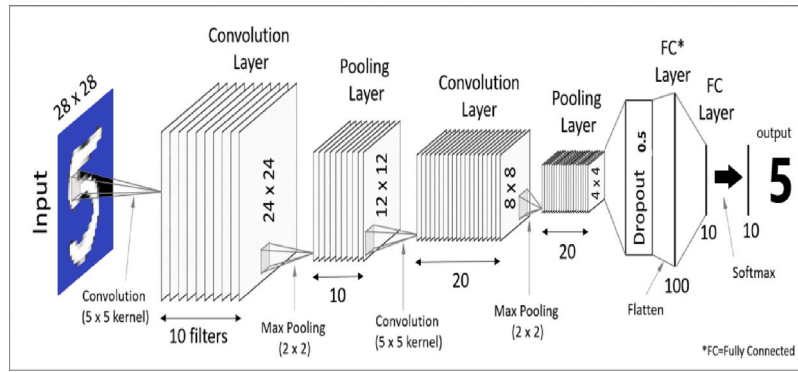


Fig. 1. Architecture of the CNN model.

Table 1
Convolutional neural network hyperparameters.

Hyperparameters	Description
Kernel(Filter) Size	The size of the kernel in a convolutional layer.
Number of feature map	The number of kernels in a convolutional layer.
Stride	Number of pixels or units that a filter or kernel is moved across the input image or feature map.
Padding	Refers to adding extra rows and columns of zeros to the input image or feature map.
Pooling type	How calculate the value for each patch on the feature map (average, Max).
Epoch	Refers to a complete iteration of the entire training dataset during the training process.
Learning rate	Amount of change in weight that is updated during learning.
Number of layers	The total number of layers that make up the network.
Number of neurons	The count of neurons in a fully-connected layer.
Batch Size	Group size to divide training data into several groups.
Weight initialization	Initialize the weights with small random numbers(Xavier initialization, He initialization).
Loss function	Function to calculate error (MSE, cross-entropy, etc.)
Optimiser	Is argument required for compiling the model (SGD, Adam, RMSprop, Adadelata, etc.)
Dropout rate	The method drops out units in neural network according to the desired probability.
Activation function	Neuron's activation function (ReLU, elu, sigmoid, etc.)

The value of the velocity for each particle is affected by three factors at the next iteration $t+1$: inertia $w \times v_{ij}(t)$, individual cognitive component $c_p \times r_p \times (pBest_{ij} - x_{ij}(t))$, and social component $c_g \times r_g \times (gBest_j - x_{ij}(t))$.

$$v_{ij}(t+1) = w \times v_{ij}(t) + c_p \times r_p \times (pBest_{ij} - x_{ij}(t)) + c_g \times r_g \times (gBest_j - x_{ij}(t)) \quad (1)$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1) \quad (2)$$

where r_p and r_g are independent and identically distributed random numbers, c_p and c_g are the coefficients on cognition and social acceleration, x_{ij} and v_{ij} are the position coordinates and velocity of the i th agent in the j th dimension, $pBest_{ij}(t)$ and $gBest_j(t)$ are the personal and global best positions in the t th iteration. The first term on the right-hand side of Eq. (1) uses w , the inertial weight, and the next two terms are suggestions for promising regions in the search space as reported by the personal and global best positions.

Once the particles have been moved, the new positions are evaluated and the two vectors $pBest_i$ and $gBest$ are updated, at iteration $t+1$ according to Eqs. (3) (in the case of maximization) and (4). This procedure is shown in algorithm 1, where N is the number of particles in the swarm.

$$pBest_i(t+1) = \begin{cases} x_i(t+1) & \text{if } f(x_i(t+1)) \geq f(pBest_i(t)) \\ pBest_i(t) & \text{otherwise} \end{cases} \quad (3)$$

$$gBest(t+1) = \underset{pBest_i}{\operatorname{argmax}}(f(pBest_i(t+1))), \quad 0 \leq i \leq N. \quad (4)$$

PSO is a metaheuristic algorithm used for solving optimization problems. It is a very good technique for solving optimization problems. Nevertheless, PSO has one drawback, which is that it gets stuck in the local minima.

Algorithm 1: PSO algorithm

```

1 for each particle in the swarm of size N do
2   - Initialize particle  $p_i$ ;
3   - Evaluate the fitness  $f(p_i)$ 
4   - Initialize  $pBest_i$  with a copy of  $p_i$ ;
5 - Calculate  $gBest$  according to (4)
6 while Maximum nb. of iterations is not reached do
7   for each particle do
8     - Calculate the particle velocity according to (1);
9     - Update the particle position according to (2);
10    - Evaluate the fitness  $f(p_i)$ 
11    - Update  $pBest_i$  according to (3);
12    - Update  $gBest$  according to (4);
13 The solution is the final global best  $gBest$ ;

```

In order to enhance the efficiency of PSO, a range of variants have been suggested by researchers. While some seek to enhance swarm initialization, others have introduced new parameters like the constriction coefficient and inertia weight. A few researchers have also experimented with different methods of defining inertia weights to optimize PSO. Additionally, some researchers have explored mutation operators to improve the global and local best particles in PSO, as seen in studies such as Imran, Hashim, and Abd Khalid (2013). PSWV, which stands for Particle Swarm Optimization without Velocity, is a variation of PSO where the equation for calculating velocity is not used.

2.3. PSWV algorithm

PSWV operates by discarding the velocity equation and instead uses a random linear combination of personal best position ($pBest$)

Algorithm 2: PSWV algorithm

```

1 for each particle in the swarm of size  $N$  do
2   - Initialize particle  $p_i$ ;
3   - Evaluate the fitness  $f(p_i)$ 
4   - Initialize  $pBest_i$  with a copy of  $p_i$ ;
5 - Calculate  $gBest$  according to (4)
6 while maximum iterations criteria is not attained do
7   for each particle do
8     - Update the particle position according to (5);
9     - Evaluate the fitness  $f(p_i)$ 
10    - Update  $pBest_i$  according to (3);
11    - Update  $gBest$  according to (4);
12 The solution is the final global best  $gBest$ ;

```

and global best position ($gBest$), as shown in Eq. (5). Consequently, particles move between their personal best position and the best global position, meaning that each particle's new position falls within the range of its personal best and global best results, this procedure is shown in algorithm 2.

Similar to PSO, the PSWV algorithm operates on a set of particles. However, it leverages an alternative mechanism to calculate the subsequent position of each particle, aiming to obtain an optimal solution in a minimal number of iterations, thereby decreasing the time required to reach the global optimum.

$$x_{ij}(t+1) = c_1 \times r_1 \times pBest_{ij}(t) + c_2 \times r_2 \times gBest_j(t) \quad (5)$$

where r_1 and r_2 are defined as the combination weights, r_1, r_2 in $[0,1]$ and c_1, c_2 in $[0,1]$ as the own and social attraction coefficients. The combination of the parameters r_1, r_2, c_1 and c_2 allows to adjust the balance between the phases of diversification and intensification of the search process.

The major advantage of using the particle swarm without velocity (PSWV) algorithm as a search algorithm is that the best CNN architecture drastically minimizes the number of iterations required to find good solutions to optimization problems (El-Sherbiny, 2011).

The results of the PSWV algorithm and the results of the different variants of particle swarm optimization were compared experimentally using the mathematical benchmark functions. The performance of the PSWV algorithm and the quality of the solution prove that PSWV is very competitive and can be considered as a viable metaheuristic algorithm for solving optimization problems.

3. The proposed algorithm

This section discusses each main component of the pswvCNN algorithm in depth.

3.1. Algorithm overview

The proposed pswvCNN method uses the PSWV algorithm to develop CNN architectures, and its framework is described in algorithm 3. Detailed flowcharts of the implementations of the pswvCNN Algorithm to optimize CNN architecture are shown in Fig. 2, which consists of three processes.

First, initialize all input parameters of the Algorithm concerning the problem at hand, including the dataset used for training and the parameters for the CNN structures to be created.

Second, initialize the population. In this step, pop_{size} particles are created randomly. Each particle has a random number of layers, and the position of the particles, the best personal position, and the best global position are initialized.

Third, at each iteration, each particle updates its position depending on its own best position, called $pBest$, in the search space and the best

Algorithm 3: Proposed pswvCNN

```

input : All parameters, Population size  $pop_{size}$ , Maximum
         number of iterations  $nb_{itrs}$ , Training data  $D_{train}$ ,  $epo_{eva}$ ;
output: The best CNN model with its architecture and selected
         hyperparameters;
1 - Initialize the swarm with a random combination of
   hyperparameters from Table 3;
2 - Train the CNN model represented by each particle in the
   population with the number of epochs  $= epo_{eva}$  and compute
   its fitness score.
3 - Initialize  $gBest = p_i$ 
4 for each particle  $p_i$  in the population do
5   - Initialize  $pBest_i$  with a copy of  $p_i$ 
6   - Calculate  $fitness(p_i.pBest)$ ,  $fitness(p_i)$ 
7   if  $fitness(gBest) \leq fitness(p_i)$  then
8      $gBest = p_i$ 
9 while Iteration criteria are not reached  $nb_{itrs}$  do
10  for each particle  $p_i$  in the population do
11    - updateParticle( $p_i$ ) ▷ algorithm 5
12    - Calculate  $fitness(p_i)$ 
13    if  $fitness(p_i) \geq fitness(p_i.pBest)$  then
14       $p_i.pBest = p_i$ 
15    if  $fitness(gBest) \leq fitness(p_i.pBest)$  then
16       $gBest = p_i.pBest$ 
17 - Save  $gBest$  and use it to reconfigure the best CNN architecture
18 - Train the best CNN architecture with the training set and
   epochs  $= epo_{train}$ 
19 - Testing the best model with the test data set

```

position in the whole population, called $gBest$, according to the update strategy proposed in Section 3.5. The process is repeated, evaluating all particles until the stopping criterion is found (in this case, it is the number of iterations).

Each particle is decoded in a CNN architecture to evaluate the particles, trained, and evaluated against the dataset. Then, the classification accuracy achieved is stored as the particle's fitness. Then update the best personal position ($pBest$) and the best global position ($gBest$).

If the maximum number of iterations is reached, $gBest$ is the best solution for our Algorithm. Otherwise, we return to the third step. The best solution is determined by this Algorithm, namely the particle represented by $gBest$, which is the best CNN architecture for this dataset.

3.2. The encoding strategy

The encoding represents different numbers of convolutional layers, pooling layers, and fully-connected layers and their hyperparameters. The encoding represents the general structure of the network (i.e., the sequence of layers) and the hyperparameters associated with each layer.

Our method uses an encoding developed directly for encoding architectures in particles. It fully describes the main representation and the hyperparameters associated with the CNN and plays an essential role in the efficiency and effectiveness of our method. This encoding defines the search space associated with the problem.

In our method, we adjust some hyperparameters listed in Table 1. We adjust the parameters concerning the structure represented by the following parameters: Number of layers, Kernel size, Number of feature maps, and Number of neurons in an Fc layer. As shown in Fig. 3, which shows the representation of the particle used in the pswvCNN algorithm, each particle has a random number of layers more significant than three and less than the predefined number len_{CNN} and

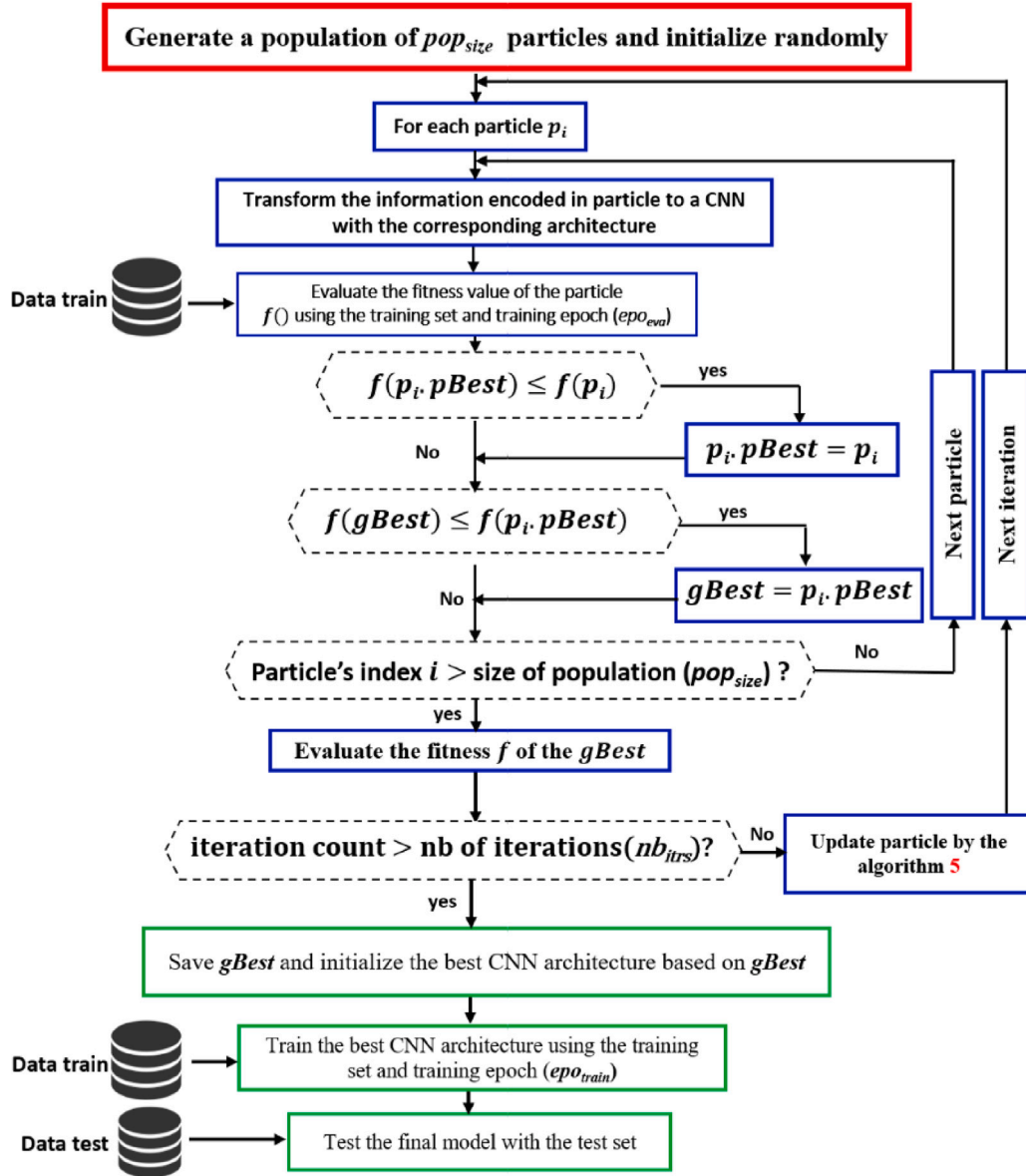


Fig. 2. Flowchart of the proposed pswvCNN algorithm.

includes three types of layers: Convolutional layers, Pooling layers, and Fully-connected layers.

To generate CNN architectures, all start with convolutional layers and end with fully-connected (Fc) layers. Convolutional layers are executed utilizing filters in a random number pattern (output feature maps) from 1 to $nb_{filters}$, and their kernel size is randomly chosen from 3×3 to $Conv_{kernel} \times Conv_{kernel}$ (7×7), where $Conv_{kernel}$ denotes the maximum convolutional kernel size and the same padding and stride parameters. The convolutional layer is always followed by a normalization operation (batch normalization Ioffe & Szegedy, 2015), and an activation function ReLu (Nair & Hinton, 2010). Each convolutional layer within the CNN, except the first, is preceded by a dropout layer. A dropout rate of 50% is near-optimal to avoid the overfitting problem (Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014).

For the pooling layer, we assume a pooling layer with a kernel size of $Pool_{kernel} \times Pool_{kernel}$ and stride of 2×2 . The number of pooling layers, num_{pool} , must satisfy the rule $num_{pool} \leq \log_2 dim$, where dim denotes the size of the input image.

Moreover, the structure of the CNN is healthy only if all fully-connected layers (Fc) are located at the end of the architecture, i.e., not

between the convolutional and pooling layers or at the beginning of the structure.

3.3. Initialization of the population

To initialize the swarm, the proposed pswvCNN creates pop_{size} particles with randomly generated CNN architectures, each particle varying between 3 layers to the maximum defined number of layers, len_{CNN} . To ensure the practicality of the generated CNN architectures, the first and last layers of each particle are always a convolutional and a fully-connected layer, respectively. Additionally, fully-connected layers cannot be positioned between convolutional or pooling layers and can only be placed towards the end of the architecture. Hence, the algorithm verifies that once a fully-connected layer is added to the architecture, all the subsequent layers are also fully-connected layers. After a certain number of pooling layers, the outputs become small enough to serve as input to the fully-connected layers without requiring many neurons.

Add convolutional layers in each particle with a random number of filters from 1 to $nb_{filters}$, and the kernel size is randomly chosen

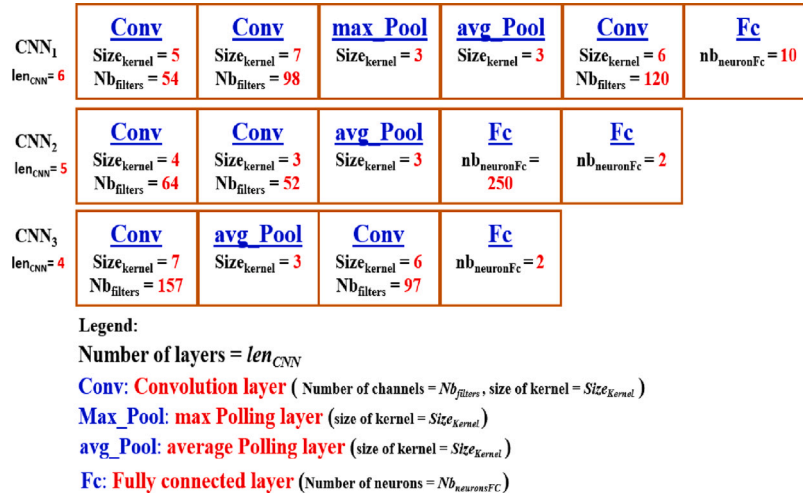


Fig. 3. Presentation of the particles in the proposed pswvCNN. Three different particles are shown.

from 3×3 to $Conv_{kernel} \times Conv_{kernel}$, the kernel strides are always 1×1 and add the activation function ReLU of all convolutional layers. Add a pooling layer or an average pooling layer with a kernel size of $Pool_{kernel} \times Pool_{kernel}$, where $Pool_{kernel} = 2$ or 3 , and a stride of 2×2 to the initial particle configuration. Add a fully-connected layer (Fc) with several neurons between 1 and $nb_{neuronFc}$ to the particle. The number of neurons in the final fully-connected layer is always equal to the number of classes in the dataset used, e.g., the MNIST dataset is ten, and the Convex dataset is 2.

3.4. Fitness evaluation

The ultimate goal of our method, as with all metaheuristic algorithms, is to find a particle with fewer errors in less time. The fitness evaluation of the proposed algorithm is shown in algorithm 4. The architecture information encoded in the particle is in the population, transferred to a CNN with the corresponding architecture, and then trained on the provided training data with epochs for particle evaluation.

We use the categorical cross-entropy loss function as a fitness criterion with the optimizer Adam (Kingma & Ba, 2014), and the weights are initialized with Xavier (Glorot & Bengio, 2010). We also use dropout, batch normalization, and regularization to avoid the problem of overfitting (Ioffe & Szegedy, 2015; Srivastava et al., 2014).

Algorithm 4: Fitness Evaluation

Input : The population, training data D_{train} , epo_{eva}
Output: The population with fitness.

- 1 **for** each particle p_i **do**
- 2 CNN $\leftarrow$ Transform the information encoded in particle to a CNN with the corresponding architecture;
- 3 Train CNN on D_{train} for a total of epo_{eva} epochs;
- 4 p_i .fitness $\leftarrow$ the classification accuracy of the trained CNN on D_{train} ;
- 5 **Return** the population .

3.5. The proposed new particle update

The process of updating the particles in the standard PSO algorithm is based on updating the velocity. However, our approach adopted the PSWV algorithm without the velocity equation. In our method, the process of velocity update was abandoned, and we adapted the algorithm to solve our problem.

In our method, updating the configuration of a particle is one of our main contributions to the proposed method. The algorithm to update

the particle is performed in *updateParticle()* in algorithm 5. This process is illustrated in Fig. 4.

We divide the particle into two parts: one part includes the conv/pool layers, and the second part includes the Fc layers, with each part handled independently.

To update a particle from the pair p_i .*pBest* and *gBest*, we search the layers of the two particles, taking one layer of *pBest* and one layer of *gBest* for each iteration, then we generate two random values λ_1 and λ_2 from $[0,1]$, where $c_1 \times r_1 = \lambda_1$ and $c_2 \times r_2 = \lambda_2$, consider that $\lambda_1 + \lambda_2 = 1$. Here, λ_1 and λ_2 stand for the probabilities of choosing the level *pBest* and *gBest*, respectively. If λ_1 is greater than λ_2 , we add layer *pBest* to the new particle, otherwise we add layer *gBest*. The iteration continues until the longest particle is exhausted.

We know that the length of the two particles p_i .*pBest* and *gBest* are not equal in all cases. To handle this case, for the particle with the smallest number of layers, we fill in the missing layers with *none* at the end of the Conv/Pool layers part or the Fc layers part.

The new particle does not contain *none* layers. This is because if the probability of the added *none* layer in one of the *gBest* or *pBest* particles is greater than the probability of the other layer, the iteration continues in this case without adding the *none* layer to the new particle.

The proposed particle update strategy allows new particles with different lengths and layers to generate the new population.

In some cases, the number of pooling layers exceeds the allowed number. The excess pooling layers are removed from the particle configuration.

The goal of introducing the particle update method, where the velocity is not updated, is to minimize the computational cost and the convergence rate (number of iterations).

4. Experimental setup

4.1. Datasets

To conduct a comprehensive evaluation of the method's effectiveness, we choose all MNIST variants: MNIST (LeCun, Bottou, Bengio, & Haffner, 1998), MNIST-RD (Larochelle, Erhan, Courville, Bergstra, & Bengio, 2007), MNIST-RB (Larochelle et al., 2007), MNIST-BI (Larochelle et al., 2007), MNIST-RD+BI (Larochelle et al., 2007). To compare the robustness of the proposed pswvCNN algorithm with that of the competitors, we also perform experiments on other datasets such as Rectangles (Larochelle et al., 2007), Rectangles-I (Larochelle et al., 2007), Convex (Larochelle et al., 2007), and MNIST-fashion (Xiao, Rasul, & Vollgraf, 2017). Fig. 5 shows examples of images in each dataset. For a description of these image datasets, see Table 2.

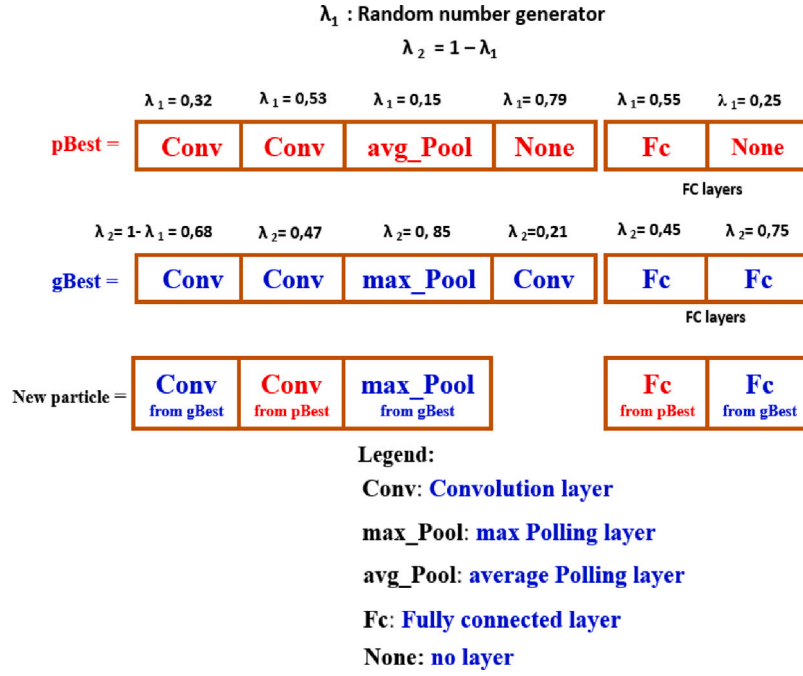


Fig. 4. Illustration of updating a particle in the proposed pswvCNN. To update a particular particle, two particles are considered: The personal best position of the particle to be updated $p_i.pBest$ and the best global particle $gBest$. The newly generated particle is created by randomly copying layers from both particles.

Algorithm 5: UpdateParticle

Input : $p_i.pBest, gBest$
Output: Updated particle

```

1 newParticle  $\leftarrow \emptyset$ 
2 for  $p_{layer}, g_{layer} \in (p_i.pBest, gBest)$  do
3    $\lambda_1 =$  Random number generator //  $\lambda_1 < 1$ ;
4    $\lambda_2 = 1 - \lambda_1$ ;
5   if  $p_{layer} \neq \text{no layer}$  then
6     if  $g_{layer} \neq \text{no layer}$  then
7       if  $\lambda_1 \geq \lambda_2$  then
8         newParticle.Add( $p_{layer}$ );
9       else
10        newParticle.Add( $g_{layer}$ );
11      end
12    else
13      if  $\lambda_1 \geq \lambda_2$  then
14        newParticle.Add( $p_{layer}$ );
15      end
16    end
17  else
18    if  $\lambda_1 < \lambda_2$  then
19      newParticle.Add( $g_{layer}$ );
20    end
21  end
22 end
```

The MNIST dataset created by LeCun et al. in 1998, is a database of handwritten digits, is a widely used collection of images that is frequently used as a benchmark in the field of machine learning. The dataset consists of a set of 70,000 images of handwritten digits, with 60,000 images for training and 10,000 images for testing. These images are black and white, have a resolution of 28 pixels by 28 pixels, and have been normalized to facilitate their use in various machine learning applications. The MNIST dataset is often used as a starting

point for building image recognition models and has become a standard benchmark for evaluating the performance of such models.

For the evaluation, we use a set of MNIST variant datasets. These datasets are more challenging to work with due to the additional distraction factors. MNIST-RD is made up of MNIST digits rotated, while MNIST-RB consists of MNIST digits with a random background. MNIST-BI shows MNIST digits against image backgrounds, while MNIST-RD+BI shows MNIST digits rotated against backgrounds. There are 12,000 training samples and 50,000 test samples for each.

In addition to MNIST and its variations, other datasets used for comparison include Convex, which contains images of geometric shapes. It has 50,000 test images and 8000 training images, and the aim is to determine whether or not the shape is convex. The Rectangles dataset includes images of rectangles of various widths and heights as well as a Rectangles-I version that features rectangles on an image background. There are 12,000 training images and 50,000 test images in the Rectangles and Rectangles-I datasets. All three datasets are two-class datasets, making them suitable for binary classification tasks.

MNIST-Fashion is a dataset of grayscale images that includes 60,000 training images and 10,000 testing images. The images are 28×28 pixels in size and depict 10 different types of fashion items, including t-shirts, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots. This dataset is commonly used for image classification tasks in machine learning and computer vision research.

4.2. Peer competitors

To evaluate the performance of the proposed pswvCNN method, we compared it with some optimized state-of-the-art CNN models. We selected competing algorithms that provide their results on the same datasets on which the proposed pswvCNN method was evaluated.

It is worth noting that several competitors have used population-based algorithms to optimize the CNN architecture, such as PSO and GA. Thus, we can conclude that the complexity of these algorithms is very similar to ours.

We used algorithms that rely on population-based metaheuristics for their strategy to compare our results: evoCNN (Sun et al., 2019), IPPSO (Wang et al., 2018), psoCNN (Junior & Yen, 2019), BQPSO (Li et al.,

Table 2
Description of datasets used in the experiments.

Dataset	Description	Input size	Classes	Training-test
MNIST	MNIST with basic digits	$28 \times 28 \times 1$	10	60,000–10,000
MNIST-BI	MNIST with background images	$28 \times 28 \times 1$	10	12,000–50,000
MNIST-RD+BI	MNIST with rotated digits and image background	$28 \times 28 \times 1$	10	12,000–50,000
MNIST-RD	MNIST with rotated digits	$28 \times 28 \times 1$	10	12,000–50,000
MNIST-RB	MNIST with random background	$28 \times 28 \times 1$	10	12,000–50,000
Convex	Discriminate between convex and concave shape	$28 \times 28 \times 1$	2	8,000–50,000
Rectangle-I	Rectangles with image background	$28 \times 28 \times 1$	2	12,000–50,000
Rectangles	Discriminate between tall and wide rectangles	$28 \times 28 \times 1$	2	1,200–50,000
MNIST-fashion	Zalando's article images	$28 \times 28 \times 1$	10	60,000–10,000



Fig. 5. Sample images of each dataset: MNIST, MNIST-BI, MNIST-RD+BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, Rectangles and MNIST-fashion.

2019), MBO-CNN (Bacanin et al., 2020a), sosCNN (Miao et al., 2021), PSO-based (Lawrence et al., 2021), MPSO-CNN (Singh et al., 2021), and IntelliSwAS (Nistor & Czibula, 2022).

In addition, other recently published algorithms with optimal parameter values that have produced promising classification errors for the given criteria are collected. These are the following: LeNet-5 (Liu et al., 2019), this is a simple CNN designed for the digit recognition problem, LSVM+RBF (Larochelle et al., 2007), SVM+Poly (Larochelle et al., 2007), CAE-1 (Rifai, Vincent, Muller, Glorot, & Bengio, 2011), CAE-2 (Rifai et al., 2011), PCANet-2 (Chan et al., 2015), RandNet-2 (Chan et al., 2015) and LDANet-2 (Chan et al., 2015).

4.3. Parameters of the algorithm

A detailed overview of the parameters used in the proposed pswCNN can be found in Table 3. It contains three categories of parameters: PSO algorithm, initialization of particles, and CNN training.

The PSO algorithm's behavior is governed by the first category of parameters, which consists of three variables: the number of executions (nb_{runs}), the number of iterations (nb_{itrs}), and the population size (pop_{size}). To guarantee reliable performance evaluation, the proposed optimization is carried out nb_{runs} times with distinct initializations. The number of iterations (nb_{itrs}) acts as the sole termination criterion and represents the total number of iterations the algorithm completes before concluding the optimization process. Additionally, the population size (pop_{size}) determines the number of particles employed by the PSO algorithm.

The second category's parameters define the initial particles (initial CNN architectures). At the initialization of each particle, each parameter is drawn randomly from its corresponding range shown in Table 3. These parameters are as follows: the number of layers (len_{CNN}), the size of the convolutional kernel ($Conv_{kernel}$), the number of filters in a convolutional layer ($nb_{filters}$), the size of the pooling kernel ($Pool_{kernel}$)

Table 3
Parameters used for evaluating the proposed pswvCNN algorithm.

Parameters	Parameters meaning	Value
psv algorithm		
nb_{runs}	Number of execution	10
nb_{lrs}	Number of iterations	10
pop_{size}	Population size	25
Initialization of particles		
len_{CNN}	Number of layers	[3–20]
$Conv_{kernel}$	Convolutional kernel size	[3–7]
$Pool_{kernel}$	Pooling kernel size	[2–3]
$nb_{filters}$	Filters number	[3–256]
$nb_{neuronFc}$	Neurons numbers in an Fc layer	[1–300]
CNNs Training		
epo_{eva}	Particle evaluation epochs	1
epo_{train}	Epochs for full training best CNN	100
$dropout$	dropout rate	0.5
$ActFun$	Activation Function	ReLU
$learningR$	learning rate	0.001
$BatchNorm$	Batch normalize layer outputs	yes

and the number of neurons in a fully-connected layer ($nb_{neuronFc}$). At the initialization of each particle, each parameter is randomly selected within the range given in the third column of Table 3, except for the output layer, which contains a fixed number of neurons set to the number of classes.

The parameters used in the third category control each particle's training process and the training process associated with the selected CNN architecture found by the optimization process. It contains the number of epochs for particle evaluation $epo_{eva}=1$, the number of epochs for evaluating the global best (i.e., the optimal architecture) $epo_{train}=100$, the dropout rate = 0.5. We emphasize that setting epo_{eva} to one was chosen to obtain an efficient optimization algorithm. As a series of experiments will show, other values higher than one can also be used. In this case, the performance may be slightly improved, but the computational cost increases.

The proposed pswvCNN method is implemented in Keras (Chollet, 2015) with Tensorflow (Abadi et al., 2016). pswvCNN runs on a computer equipped with an Nvidia GeForce GTX1070 model GPU card.

5. Results and analysis

5.1. Results

The following section displays the results and compares our method with other methods regarding error rates on nine datasets. We also show the optimal architectures obtained with our methods.

Since our approach is based on a metaheuristic algorithm, multiple runs of an experiment and statistical analysis are required to draw a conclusion and investigate convergence to the optimal solution. In our experiments, ten runs are performed due to the limited computational resources.

The experimental results of the proposed pswvCNN algorithm are presented in Table 4 (in the last three rows), including the best error rate, the mean error rate, and the standard deviations (std dev) of the classification errors from the ten runs. To make the results of the comparison explicit, the symbols (+) and (–) are inserted in the table next to the error obtained by the competing methods. These symbols indicate that the result of pswvCNN is significantly better or worse than the best result of the corresponding competitor; the symbol (=) indicates that the best error rate of pswvCNN is similar to that of the competitor. The symbol (–) indicates no results for the competitor.

The best test errors obtained by the proposed pswvCNN for the eight datasets: MNIST, MNIST-BI, MNIST-RD+BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, and Rectangles, shown in Table 4, are 0.30%, 2.27%, 11.47%, 2.74%, 1.67%, 1.35%, 1.33%, and 0.01%, respectively.

While the average test errors for 10 runs on these datasets are 0.44%, 2.75%, 14.58%, 3.69%, 2.32%, 2.78%, 1.81%, and 0.78%, with standard deviation of 0.04%, 0.37%, 2.43%, 0.62%, 0.37%, 0.92%, 0.82%, and 0.92%, respectively. Fig. 6 shows a boxplot of the test accuracy distribution (over ten runs) for all datasets.

For MNIST, the best CNN model using our proposed method achieves the best error rate of 0.30%, compared with the best error rate of 0.32% of psoCNN and 0.35% of PSO-based, which is only slightly larger than 0.11% of MPso-CNN. Our findings using simple CNN architectures are among the top-performing results on the MNIST dataset. The current best reported test error of 0.21% was achieved using neural networks and DropConnect technique (Wan, Zeiler, Zhang, Le Cun, & Fergus, 2013).

Regarding the dataset MNIST-BI, the best CNN model developed with our proposed method achieves the best error rate of 2.27% with a mean of 2.75%. BQ-CNN produced the best model for this dataset with an error rate of 0.95% and a mean of 0.95%, followed by the two methods, psoCNN and PSO-based, with error rates of 1.90% and 2.20%, respectively.

Again, for MNIST-RD and MNIST-RB, BQ-CNN is the primary reference method with the best error rate of 0.98% and 0.97%, respectively. Our proposed method provides the best error rate of 2.74% and 1.67%, respectively, and outperforms psoCNN, PSO-based and other methods.

For MNIST-RD+BI, we obtain a best-estimated error rate of 11.47%, which is better than the results of all methods IPPSO, psoCNN, BQ-CNN, PSO-based and IntelliSwAS, which obtained: 34.50%, 14.28%, 14.20%, 11.61%, and 15.74%, respectively.

Our proposed method achieves an error rate of 0.72% for the Rectangles-I dataset, which outperforms all other competing methods. Moreover, for the Rectangles dataset, our proposed method produces the best CNN model with an error rate of 0.01%, while the PSO-based and IntelliSwAS methods achieve an error rate of 0.0%.

For the Convex dataset, our CNN model provides an error rate of 1.35%, outperforming the models generated by psoCNN and PSO-based models, which achieve error rates of 1.70% and 1.36%, respectively.

We used the MNIST-Fashion dataset to compare the results in terms of the number of trainable parameters for the model resulting from using our method with other models. The results for the MNIST-Fashion dataset are shown in Table 5. It can be seen that the proposed pswvCNN method outperforms all competitors when comparing the best performance. The best test error achieved is 5.44%. The best performance of pswvCNN is even better than that of fourteen competitors. Fig. 7 illustrates the evolution of the best particle score (training accuracy) for the ten runs. It also shows the boxplot of the test accuracies obtained by the best particles.

5.2. Discussion

The proposed algorithm has the capability to discover smaller CNN architectures than other competing models due to its initialization with a compact architecture having a length ranging from 3 to 20 layers. Hand-designed networks often have redundant parameters, while the most sophisticated models can be represented by smaller networks. With the proposed algorithm, populations are generated from networks of various lengths, but smaller networks converge faster compared to extensive ones.

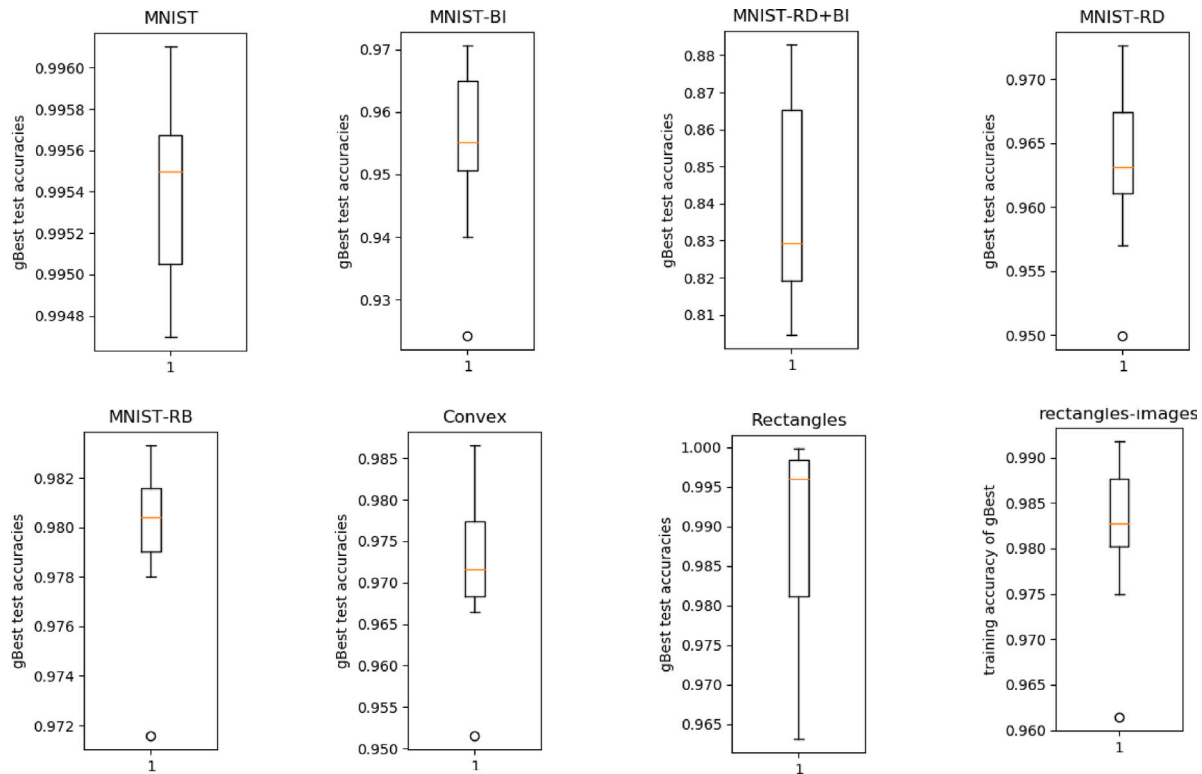
In six of the nine datasets included in Table 2, our pswvCNN algorithm-designed models outperform the most recent models. The models resulting from pswvCNN have much fewer parameters than well-known models such as VGG16, GoogleNet, AlexNet, and evoCNN, as shown in Table 7. For example, the best model found for the MNIST-fashion dataset contains 5.15 million parameters.

To diagnose the performance of the CNN model obtained by our method, we show in Fig. 8 the training curves of the best model found by pswvCNN on the Convex dataset for 100 epochs. The test dataset is used as a validation set. The plot of the learning curves shows a

Table 4

Comparison of error rate of pswvCNN and different models”.

Model/Dataset	MNIST	MNIST-BI	MNIST-RD+BI	MNIST-RD	MNIST-RB	Convex	Rectangles-I	Rectangles
LeNet-5 (Lecun et al., 1998)	0.95%(+)	–	–	–	–	–	–	–
SVM+RBF (Larochelle et al., 2007)	3.03%(+)	22.61%(+)	32.62%(+)	10.38%(+)	14.58%(+)	19.13%(+)	–	–
SVM+Poly (Larochelle et al., 2007)	3.69%(+)	24.01%(+)	37.59%(+)	13.61%(+)	16.62%(+)	19.82%(+)	–	–
CAE-1 (Rifai et al., 2011)	2.83%(+)	16.7%(+)	48.10%(+)	11.59%(+)	13.57%(+)	–	21.86%(+)	1.48%(+)
CAE-2 (Rifai et al., 2011)	2.48%(+)	15.5%(+)	45.23%(+)	9.66%(+)	10.90%(+)	–	21.54%(+)	1.21%(+)
PCANet-2 (Chan et al., 2015)	1.06%(+)	11.55%(+)	35.86%(+)	8.52%(+)	6.85%(+)	4.19%(+)	13.39%(+)	0.49%(+)
RandNet-2 (Chan et al., 2015)	1.27%(+)	11.65%(+)	43.69%(+)	8.47%(+)	13.47%(+)	5.45%(+)	17.00%(+)	0.09%(+)
LDANet-2 (Chan et al., 2015)	1.40%(+)	12.42%(+)	38.54%(+)	4.52%(+)	6.81%(+)	7.22%(+)	16.20%(+)	0.14%(+)
IPPSO (Wang et al., 2018)	1.13%(+)	–	34.50%(+)	–	–	8.48%(+)	–	–
evocNN (Sun et al., 2019)	1.18%(+)	4.53%(+)	35.03%(+)	5.22%(+)	2.80%(+)	4.82%(+)	5.03%(+)	0.01%(=)
psoCNN (Junior & Yen, 2019)	0.32%(+)	1.90%(–)	14.28%(+)	3.58%(+)	1.79%(+)	1.70%(+)	2.22%(+)	0.03%(+)
BQ-CNN (Li et al., 2019)	0.99%(+)	0.95%(–)	14.20%(+)	0.98%(–)	0.97%(–)	1.65%(+)	–	–
MBO-CNN (Bacanin et al., 2020a)	0.36%(+)	–	–	–	–	–	–	–
sosCNN (Miao et al., 2021)	0.38%(+)	1.68%(–)	10.65%(–)	3.01%(+)	1.49%(–)	1.4%(+)	1.57%(+)	0%(–)
PSO-based (Lawrence et al., 2021)	0.35%(+)	2.2%(–)	11.61%(+)	3.23%(+)	1.8%(+)	1.36%(+)	1.01%(+)	0%(–)
MPSo-CNN (Singh et al., 2021)	0.11%(–)	–	31.9%(+)	–	–	7.27%(+)	–	–
IntelliSwAS (Nistor & Czibula, 2022)	0.38%(+)	3.53%(+)	15.74%(+)	4.46%(+)	2.55%(+)	1.20%(+)	2.22%(+)	0%(–)
pswvCNN (best)	0.30%	2.27%	11.47%	2.74%	1.67%	1.35%	0.72%	0.01%
pswvCNN (mean)	0.44%	2.75%	14.58%	3.69%	2.32%	2.78%	1.81%	0.78%
pswvCNN (std dev)	0.04%	0.37%	2.43%	0.62%	0.37%	0.92%	0.82%	0.92%

**Fig. 6.** Boxplots of the gBest test accuracy for all datasets obtained with the proposed pswvCNN. Ten runs were performed.

good fit: the training loss curve decreases to the point of stability. The curve of validation losses decreases up to the point of stability and has a small distance from the training losses. The proposed pswvCNN can find suitable models for a given image classification dataset.

As for the depth of the models obtained by the pswvCNN algorithm, compared to the ones produced by other benchmark models IPPSO, psoCNN, PSO-based, and sosCNN, as shown in Fig. 9, although there were no bars for some methods in some of the datasets, the depths of the best networks formed by pswvCNN are compared to those of the benchmark models IPPSO, psoCNN, PSO-based, and sosCNN. We obtained By our method the following depths of CNN architectures: 7, 7, 8, 7, 7, 8, 11, 8, and 7 layers for the following datasets respectively: MNIST, MNIST-BI, MNIST-RD+BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, Rectangles and MNIST-fashion. The experimental

findings show that pswvCNN is capable of designing deeper network architectures. On the other hand, some competing methods, tend to generate shallower networks, i.e., their created networks are limited in terms of maximum depths due to vanishing gradients. The vanishing gradient problem, in particular, has an influence on a model's capacity to learn. Backpropagation is unable to modify weights when the gradients approach zero in this circumstance, and learning ceases.

The network architectures obtained with the sosCNN algorithm consistently exhibit greater depth in all datasets, which can be attributed to the ability of the slack-gain strategy to obtain deeper network architectures.

The PSO algorithm is executed at each iteration to reevaluate the particles and compute the fit to find the $pBest$ for each particle and $gBest$ for the population. Since the particles are convolutional neural

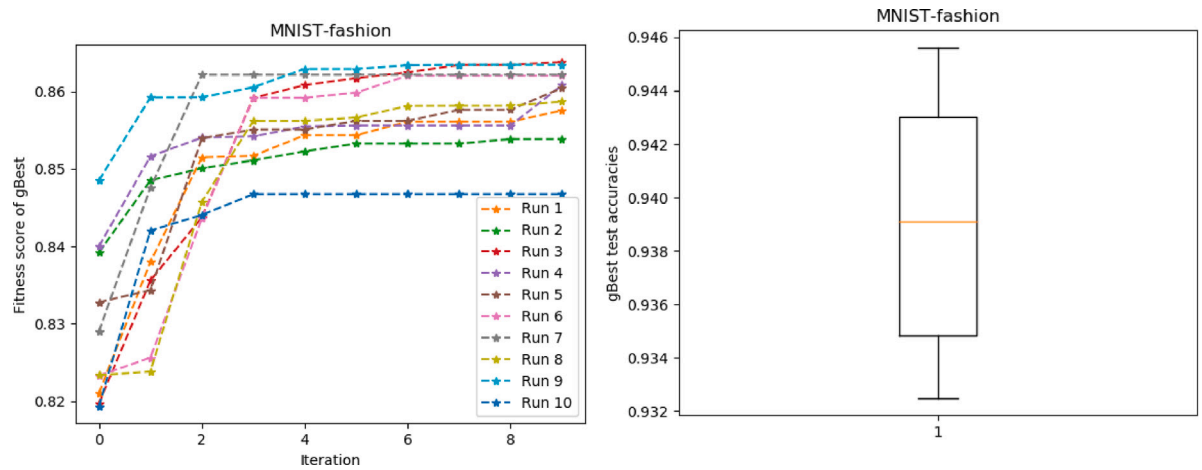


Fig. 7. Left: Training accuracy (fitness score) of the best particle at each iteration (with epo_{eva} epochs). The MNIST-fashion dataset was used. Ten runs were performed. Right: Boxplot for test accuracy for the best model found by pswvCNN.

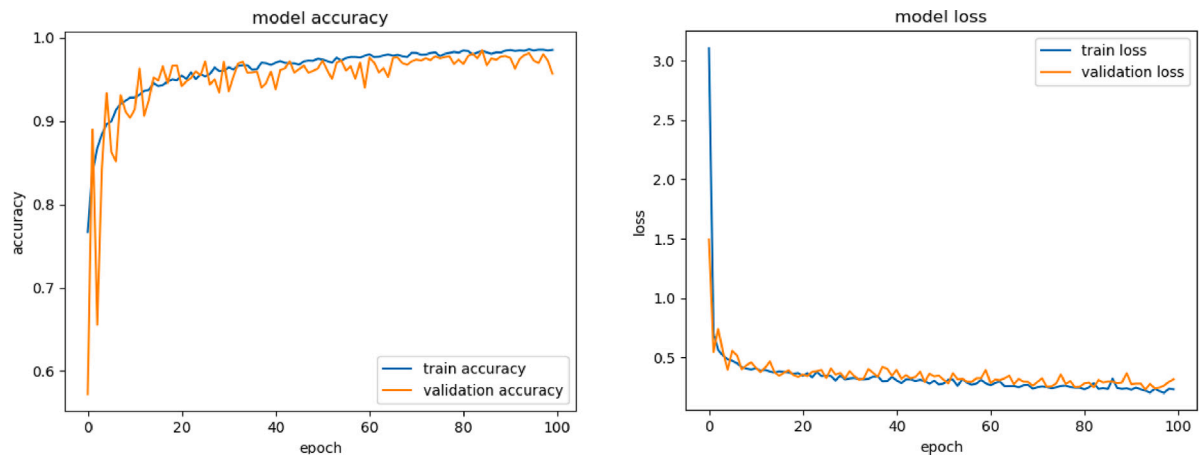


Fig. 8. The performance plot of the most optimal model discovered by pswvCNN on the Convex dataset during training. Left: model accuracy, Right: model loss.

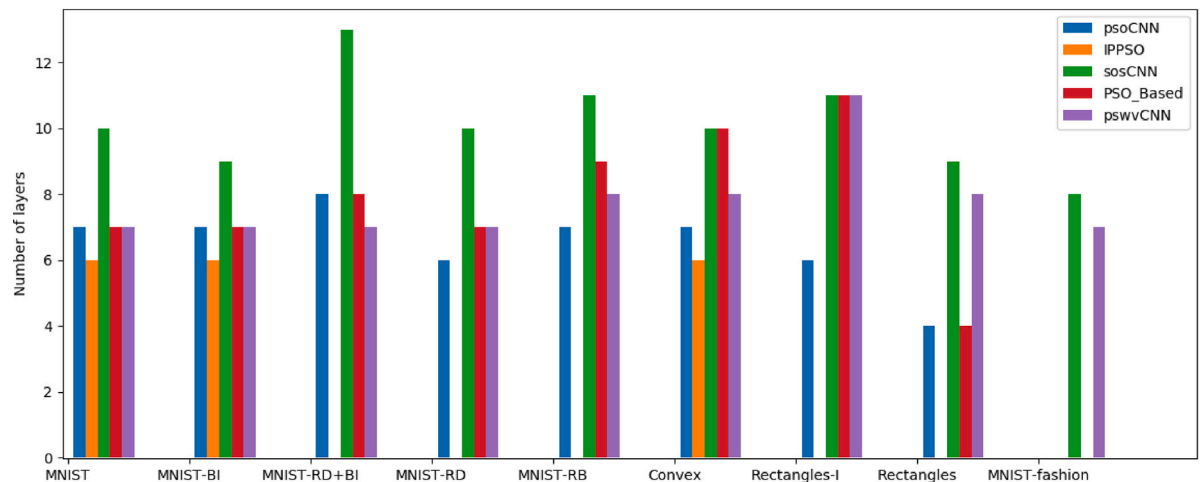


Fig. 9. A comparison of model depths between the networks devised by the benchmark models and our methods across all datasets.

networks, their evaluation depends on the accuracy value obtained by training them for the number of epochs epo_{eva} . This necessitates a sufficient but not excessive amount. If a high number of epochs is used to evaluate a CNN model, it may take longer to discover the optimal architecture as each particle has to be trained for the same number of epochs before its performance can be assessed. Fig. 10 shows the effects

of using 1, 3, and 7 epochs while evaluating particles. The larger the value of epo_{eva} , the more accurate the evaluation, but it takes a long time. For example, with the Convex dataset, when we use $epo_{eva}=1$, we get the best error rate of the best model for ten runs of our method 1.35%, with a runtime of 0.49 h, and when using $epo_{eva}=3$ the best error rate for ten executions is 1.28% at a time of 4.33 h. Also, when

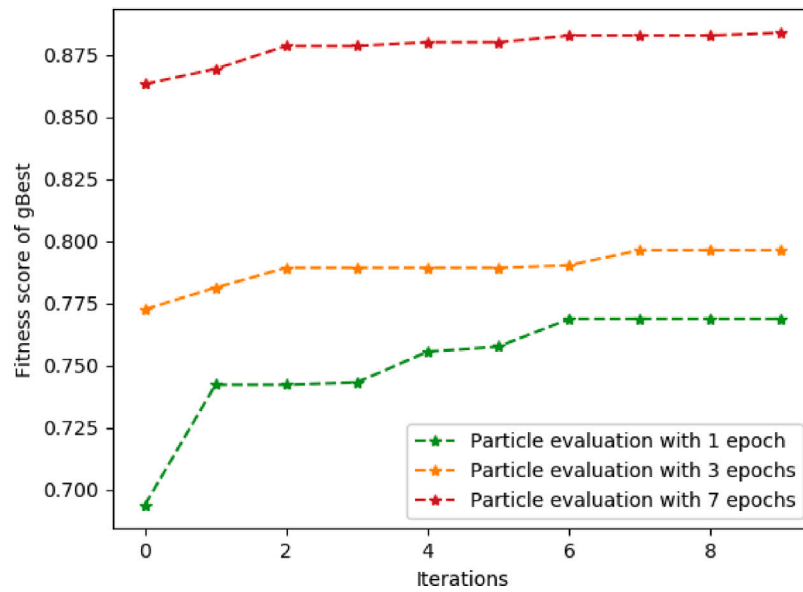


Fig. 10. Influence of the number of epochs value used to evaluate the particles on the convex dataset.

Table 5
PswvCNN test errors for the MNIST-Fashion dataset compared to different models.

Model	Test error (%)	#Parameters	#Epoch
2C1P2F+Drouout (Sun et al., 2019)	8.40(+)	3.27 M	300
2C1P (Sun et al., 2019)	7.50(+)	100 K	30
3C2F (Sun et al., 2019)	9.30(+)	–	–
3C1P2F+Dropout (Sun et al., 2019)	7.40(+)	7.14 M	150
GRU+SVM+Dropout (Sun et al., 2019)	10.30(+)	–	100
SqueezeNet-200 (Iandola et al., 2016)	10.00(+)	500 K	200
MLP 256-128-64 (Sun et al., 2019)	10.00(+)	41 K	25
AlexNet (Krizhevsky, Sutskever, & Hinton, 2012)	10.1%(+)	62.3 M	–
VGG16 (Simonyan & Zisserman, 2014)	6.5%(+)	26 M	–
GoogLeNet (Szegedy et al., 2015)	6.3%(+)	23 M	200
evoCNN (best) (Sun et al., 2019)	5.47%(+)	6.68 M	100
evoCNN (mean) (Sun et al., 2019)	7.28%(+)	6.52 M	100
psoCNN (best) (Junior & Yen, 2019)	5.53%(+)	2.32 M	200
psoCNN (mean) (Junior & Yen, 2019)	5.90%(+)	2.5 M	200
sosCNN (best) (Miao et al., 2021)	5.68%(+)	2.30 M	100
sosCNN (mean) (Miao et al., 2021)	6.17%(+)	3.42 M	100
pswvCNN (best)	5.44%	5.15 M	100
pswvCNN (mean)	6.1%	3.74 M	100
pswvCNN (std dev)	0.46%	0.81 M	100

Table 6
Wilcoxon signed ranks test results.

Comparison	Statistic	P-value
pswvCNN versus psoCNN (Junior & Yen, 2019)	5.0	0.068
pswvCNN versus sosCNN (Miao et al., 2021)	18.0	1.0
pswvCNN versus IntelliSwAS (Nistor & Czibula, 2022)	4.0	0.049
pswvCNN versus PSO-based (Lawrence et al., 2021)	5.0	0.068
pswvCNN versus evoCNN (Sun et al., 2019)	5.0	0.068

implementing the algorithm pswvCNN by changing the value of epo_{eva} , we noticed that the depth of the resulting models (number of layers) is directly proportional to the value of epo_{eva} .

Fig. 11 shows the evolution of the fitness score of $gBest$ for ten different runs with ten iterations each on all datasets with 25 particles, and all other parameters are listed in Table 3. We can see that the accuracy is continuously improving. We note that the convergence is fast. At the beginning of the evolutionary process, the accuracy of the $gBest$ particles increases rapidly until the second iteration. After that, the particles continue to search but struggle to find better solutions until the tenth iteration. It can be seen that the algorithm converges

rapidly due to the efforts to minimize the search space in the proposed coding strategy and the particle update strategy.

The Wilcoxon signed-rank test was conducted to compare the performance of six different models: pswvCNN, psoCNN, sosCNN, IntelliSwAS, PSO-based, and evoCNN Table 6. The test aimed to determine whether there is a significant difference in performance between these models. To compare the performance of our method pswvCNN with the best methods from psoCNN, sosCNN, IntelliSwAS, PSO-based, and evoCNN, which have results for all eight (08) datasets.

For each comparison, the test produced a test statistic and a p -value. The test statistic represents the size of the difference between the two groups being compared (the p -values and test statistic have been computed by using the package scipy in python). A larger test statistic indicates a larger difference. The p -value represents the probability of observing a test statistic as large as the one obtained, assuming that there is no significant difference between the groups.

– pswvCNN and psoCNN have a Wilcoxon statistic of 5.0 and a p -value of 0.0687. This suggests that there is not enough evidence to reject the null hypothesis that the two models have the same performance, although the p -value is relatively close to the conventional threshold of 0.05.

– The results demonstrate that there is a significant difference in performance between pswvCNN and IntelliSwAS, with a p -value of 0.049. This indicates that pswvCNN significantly outperforms IntelliSwAS. However, there is no significant difference in performance between pswvCNN and sosCNN, with a p -value of 1.0. This implies that there is insufficient evidence to conclude that one model performs better than the other.

Furthermore, the results suggest that there is a marginal difference in performance between pswvCNN and the other models (psoCNN, PSO-based, and evoCNN), with a p -value of 0.069 for each comparison. It can be concluded that there might be a variation in the performance of pswvCNN and the other models.

Table 7 shows, for each dataset, the best CNN architecture with their layers and the parameters for each layer, and in the next column, the number of trainable parameters for each dataset. We can observe that the algorithm favors using straightforward structures with high potential for performance. This is important because recent research has confirmed that a single (Springenberg, Dosovitskiy, Brox, & Riedmiller, 2014).

One of our most important contributions in this area is to reduce the execution time of pswvCNN. Due to reducing particle updates by

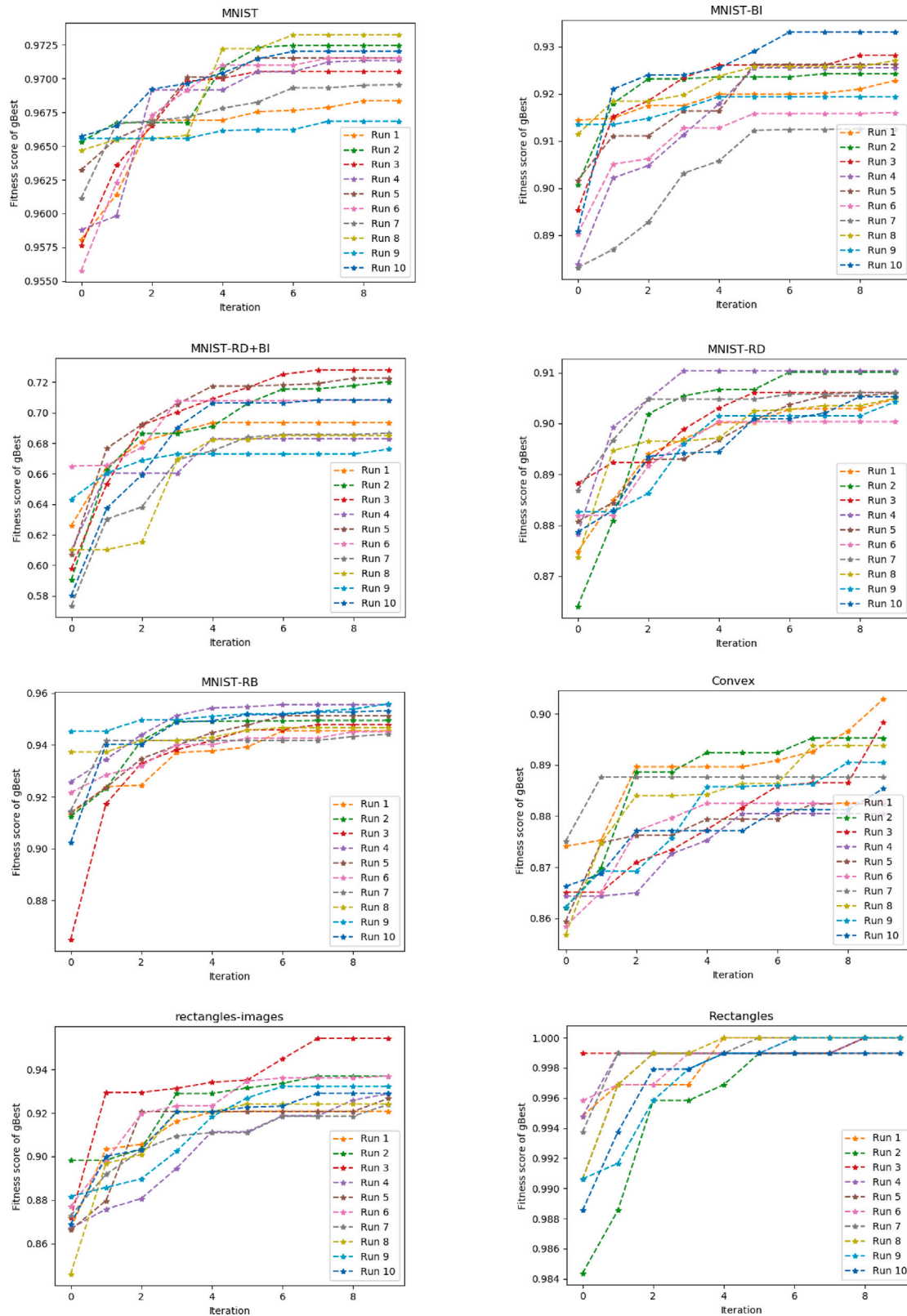


Fig. 11. Plotting the evolution of training accuracy using the proposed pswvCNN for all datasets over 10 iterations in 10 runs.

abandoning the velocity equation, our results show that pswvCNN can find highly competitive CNN architectures faster. The average execution time for all datasets: MNIST, MNIST-BI, MNIST-RD + BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, Rectangles datasets, and MNIST-fashion are: 7 h, 6 h, 3.88 h, 3.7 h, 0.49 h, 2.42 h, 0.7 h, and

10 h respectively, determined on a computer with an Nvidia GeForce GTX 1070 GPU card.

Despite the small size of the datasets used in our experiments, our results indicate that pswvCNN can effectively search for optimized CNN architectures. Therefore, pswvCNN holds promise as a tool to aid

Table 7

Best CNN architectures found by pswvCNN for each dataset: MNIST, MNIST-BI, MNIST-RD+BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, Rectangles, and MNIST-fashion.

Dataset	Layer	Parameters	# Parameters
MNIST	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 184$	3.64 M
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 251$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 214$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 204$	
	Max Pooling	$Pool_{kernel}: 3 \times 3;$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 102$	
	Fully-Connected	$nb_{neuronFc}: 10$	
MNIST-BI	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 166$	3.6 M
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 116$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 185$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 132$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 132$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 132$	
	Fully-Connected	$nb_{neuronFc}: 10$	
MNIST-RD+BI	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 111$	5.4 M
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 186$	
	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 222$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 197$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 207$	
	Average Pooling	$Pool_{kernel}: 3 \times 3;$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 213$	
MNIST-RD	Fully-Connected	$nb_{neuronFc}: 10;$	3.38 M
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 250$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 65$	
	Average Pooling	$Pool_{kernel}: 3 \times 3;$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 215$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 215$	
	Average Pooling	$Pool_{kernel}: 3 \times 3$	
MNIST-RB	Fully-Connected	$nb_{neuronFc}: 10;$	5.3 M
	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 191$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 107$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 157$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 223$	
	Average Pooling	$Pool_{kernel}: 3 \times 3$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 208$	
Convex	Fully-Connected	$nb_{neuronFc}: 2;$	3.6 M
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 222$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 143$	
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 149$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 220$	
	Average Pooling	$Pool_{kernel}: 2 \times 2;$	
	Max Pooling	$Pool_{kernel}: 3 \times 3$	
Rectangles-I	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 123$	2.76 M
	Fully-Connected	$nb_{neuronFc}: 2;$	
	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 239$	
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 147$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 182$	
	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 130$	
	Convolution	$Conv_{kernel}: 3 \times 3; nb_{filters}: 97$	
Rectangles	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 249$	3.11 M
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 66$	
	Average Pooling	$Pool_{kernel}: 2 \times 2$	
	Average Pooling	$Pool_{kernel}: 2 \times 2$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 150$	
	Fully-Connected	$nb_{neuronFc}: 2;$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 136$	
MNIST-fashion	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 139$	5.15 M
	Max Pooling	$Pool_{kernel}: 3 \times 3;$	
	Convolution	$Conv_{kernel}: 5 \times 5; nb_{filters}: 150$	
	Convolution	$Conv_{kernel}: 6 \times 6; nb_{filters}: 225$	
	Max Pooling	$Pool_{kernel}: 3 \times 3$	
	Convolution	$Conv_{kernel}: 4 \times 4; nb_{filters}: 244$	
	Fully-Connected	$nb_{neuronFc}: 2;$	

(continued on next page)

Table 7 (continued).

Dataset	Layer	Parameters	# Parameters
	Convolution	$Conv_{kernel}: 7 \times 7; nb_{filters}: 188$	
	Max Pooling	$Pool_{kernel}: 3 \times 3$	
	Fully-Connected	$nb_{neuronFc}: 10;$	

deep learning practitioners in the automated design of CNN models for diverse domain-specific applications.

6. Conclusion, limitations and future work

In summary, in this paper we present an approach to find the best CNN architecture by implementing a PSO algorithm without velocity equation applied to image classification. The main goal of our method is to find hyperparameters that build a CNN, such as the total number of layers, the number of convolutional layers, the number of pooling layers, the size of the kernel used in each convolutional layer, the number of convolutional filters, and the number of neurons in each Fc layer.

The outcomes of our study demonstrate that pswvCNN can rapidly identify an optimized CNN architecture for any provided dataset. Using just 25 particles and 10 iterations, the algorithm can produce models that achieve test errors similar to those obtained by more complex and sophisticated designs. Our experimental findings suggest that pswvCNN has the potential to discover even better architectures with more computational resources at its disposal.

This study presents an Evolutionary Computation (EC) approach for designing CNN architectures without any human intervention or prior knowledge of CNNs or the target problem. The proposed method surpasses both EC and non-EC counterparts on frequently used benchmark datasets such as MNIST, MNIST-BI, MNIST-RD + BI, MNIST-RD, MNIST-RB, Convex, Rectangles-I, Rectangles, and MNIST-fashion.

In this study, we developed a simple and effective strategy for autonomously generating CNN architectures in a short time, eliminating the limits of the traditional PSO search process. This goal was achieved by bringing about a change in the traditional PSO algorithm, which is to dispense with the velocity equation in the traditional PSO algorithm, and the effectiveness and efficiency of the proposed pswvCNN method have been demonstrated by achieving the most competitive accuracy and robustness, also outperforming most selected competitors in different standard datasets.

The pswvCNN algorithm outperforms all algorithms studied on many datasets because it is less prone to premature convergence and less likely to get stuck in local optima. Our algorithm can converge to the global optimum faster and allow particles trapped in a local optimum to escape it. This is due to the idea of allowing the best particles to share their information with all other particles and update their positions.

The proposed strategy outperformed the competitors in terms of classification accuracy, model size obtained, and search time for the final CNN design.

From the experiments and the results of the pswvCNN optimization methodology, we can conclude that the accuracy has improved in all the case studies performed and a robust performance has been achieved with the fewest trainable parameters in a shorter time.

One limitation of this study is that its search strategies are unable to create fresh layer configurations that possess new parameters. This is because the particle position is updated through random duplication of layers obtained from improved personal or global solutions. This suggests that the update mechanisms in our method lack optimal diversity and condensation. Nevertheless, the results are impressive. To address this problem in future work, it is beneficial to develop an adaptive strategy. We propose the inclusion of a mutation strategy in the updating process of the swarm particles to achieve a stronger exploration capability and a fast convergence rate.

As future work, it would be interesting to extend our research to larger datasets such as ImageNet and CIFAR-100 to test the algorithm more comprehensively. In addition, we plan to investigate all other CNN hyperparameters, including the number of epochs, learning rate, activation function, optimizer, dropout rate, convolutional layer step size (stride), padding type, and others. We will also explore evolutionary techniques for recurrent neural networks (RNN) that help deal with time-varying inputs such as speech and video data. We also want to investigate whether it is possible to work with input images in real time or capture them via video. We also want to explore the development of the pswvCNN algorithm to create parallel CNN architectures that can handle more complex CNNs.

Our efforts will also focus on defining a non-PSO metaheuristic algorithm or hybrid approach capable of efficiently generating optimized solutions in a short time frame.

CRedit authorship contribution statement

D. Elhani: Software, Validation, Data curation, Investigation, Writing – original draft, Writing – review & editing. **A.C. Megherbi:** Software, Validation, Investigation, Writing – original draft, Writing – review & editing. **A. Zitouni:** Software, Validation, Investigation, Writing – original draft, Writing – review & editing. **F. Dornaika:** Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Supervision. **S. Sbaa:** Investigation, Writing – original draft, Writing – review & editing. **A. Taleb-Ahmed:** Investigation, Writing – original draft, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)* (pp. 265–283).
- Aszemi, N. M., & Dominic, P. (2019). Hyperparameter optimization in convolutional neural network using genetic algorithms. *International Journal of Advanced Computer Science and Applications*, 10(6), 269–278.
- Bacanin, N., Bezdan, T., Tuba, E., Strumberger, I., & Tuba, M. (2020a). Monarch butterfly optimization based convolutional neural network design. *Mathematics*, 8(6), 936.
- Bacanin, N., Bezdan, T., Tuba, E., Strumberger, I., & Tuba, M. (2020b). Optimizing convolutional neural network hyperparameters by enhanced swarm intelligence metaheuristics. *Algorithms*, 13(3), 67.
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(2).
- Chan, T. H., Jia, K., Gao, S., Lu, J., Zeng, Z., & Ma, Y. (2015). PCANet: A simple deep learning baseline for image classification? *IEEE Transactions on Image Processing*, 24(12), 5017–5032.
- Chollet, F. (2015). Keras. <https://github.com/fchollet/keras>.
- El-Sherbiny, M. M. (2011). Particle swarm inspired optimization algorithm without velocity equation. *Egyptian Informatics Journal*, 12(1), 1–8.
- Fregoso, J., Gonzalez, C. I., & Martinez, G. E. (2021). Optimization of convolutional neural networks architectures using pso for sign language recognition. *Axioms*, 10(3), 139.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249–256). JMLR Workshop and Conference Proceedings.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition (2015). cite. arXiv Preprint arXiv:1512.03385.
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 7132–7141).
- Hu, X. M., Zhang, S. R., Li, M., & Deng, J. D. (2021). Multimodal particle swarm optimization for feature selection. *Applied Soft Computing*, 113, Article 107887.
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *2017 IEEE conference on computer vision and pattern recognition* (pp. 2261–2269). <http://dx.doi.org/10.1109/CVPR.2017.243>.
- Hubel, D. H., & Wiesel, T. N. (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of Physiology*, 160(1), 106.
- Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., & Keutzer, K. (2016). SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size. arXiv preprint arXiv:1602.07360.
- Imran, M., Hashim, R., & Abd Khalid, N. E. (2013). An overview of particle swarm optimization variants. *Procedia Engineering*, 53, 491–496.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448–456). PMLR.
- Junior, F. E. F., & Yen, G. G. (2019). Particle swarm optimization of deep neural networks architectures for image classification. *Swarm and Evolutionary Computation*, 49, 62–74.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, vol. 4 (pp. 1942–1948). IEEE.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv Preprint arXiv:1412.6980.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 84–90.
- Larochelle, H., Erhan, D., Courville, A., Bergstra, J., & Bengio, Y. (2007). An empirical evaluation of deep architectures on problems with many factors of variation. In *Proceedings of the 24th international conference on machine learning* (pp. 473–480).
- Lawrence, T., Zhang, L., Lim, C. P., & Phillips, E. J. (2021). Particle swarm optimization for automatically evolving convolutional neural networks for image classification. *IEEE Access*, 9, 14369–14386.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Li, M., Du, W., & Nian, F. (2014). An adaptive particle swarm optimization algorithm based on directed weighted complex network. *Mathematical Problems in Engineering*, 2014, 1–7.
- Li, Y., Xiao, J., Chen, Y., & Jiao, L. (2019). Evolving deep convolutional neural networks by quantum behaved particle swarm optimization with binary encoding for image classification. *Neurocomputing*, 362, 156–165.
- Liu, X., Deng, Z., & Yang, Y. (2019). Recent progress in semantic image segmentation. *Artificial Intelligence Review*, 52(2), 1089–1106.
- Lorenzo, P. R., Nalepa, J., Ramos, L. S., & Pastor, J. R. (2017). Hyper-parameter selection in deep neural networks using parallel particle swarm optimization. In *Proceedings of the genetic and evolutionary computation conference companion* (pp. 1864–1871).
- Loussaief, S., & Abdelkrim, A. (2018). Convolutional neural network hyper-parameters optimization based on genetic algorithms. *International Journal of Advanced Computer Science and Applications*, 9(10), 252–266.
- Martinez-Cantin, R. (2014). BayesOpt: a Bayesian optimization library for nonlinear optimization, experimental design and bandits. *Journal of Machine Learning Research*, 15(1), 3735–3739.
- Miao, F., Yao, L., & Zhao, X. (2021). Evolving convolutional neural networks by symbiotic organisms search algorithm for image classification. *Applied Soft Computing*, 109, Article 107537.
- Mosa, M. A. (2020). A novel hybrid particle swarm optimization and gravitational search algorithm for multi-objective optimization of text mining. *Applied Soft Computing*, 90, Article 106189.
- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *ICML*.
- Nistor, S. C., & Czibula, G. (2022). IntelliSwAS: Optimizing deep neural network architectures using a particle swarm-based approach. *Expert Systems with Applications*, 187, Article 115945.
- Rifai, S., Vincent, P., Muller, X., Glorot, X., & Bengio, Y. (2011). Contractive auto-encoders: Explicit invariance during feature extraction. In *ICML*.
- Sibali, T. V. (2019). Particle swarm optimisation in designing parameters of manufacturing processes: A review (2008–2018). *Applied Soft Computing*, 84, Article 105743.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- Singh, P., Chaudhury, S., & Panigrahi, B. K. (2021). Hybrid MPSO-CNN: Multi-level particle swarm optimized hyperparameters of convolutional neural network. *Swarm and Evolutionary Computation*, 63, Article 100863.
- Springenberg, J. T., Dosovitskiy, A., Brox, T., & Riedmiller, M. (2014). Striving for simplicity: The all convolutional net. arXiv preprint arXiv:1412.6806.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- Strumberger, I., Tuba, E., Bacanin, N., Zivkovic, M., Beko, M., & Tuba, M. (2019). Designing convolutional neural network architecture by the firefly algorithm. In *2019 International young engineers forum* (pp. 59–65). IEEE.

- Suganuma, M., Shirakawa, S., & Nagao, T. (2017). A genetic programming approach to designing convolutional neural network architectures. In *Proceedings of the genetic and evolutionary computation conference* (pp. 497–504).
- Sun, Y., Xue, B., Zhang, M., & Yen, G. G. (2019). Evolving deep convolutional neural networks for image classification. *IEEE Transactions on Evolutionary Computation*, 24(2), 394–407.
- Sun, Y., Xue, B., Zhang, M., Yen, G. G., & Lv, J. (2020). Automatically designing CNN architectures using the genetic algorithm for image classification. *IEEE Transactions on Cybernetics*, 50(9), 3840–3854.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., et al. (2015). Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1–9).
- Wan, L., Zeiler, M., Zhang, S., Le Cun, Y., & Fergus, R. (2013). Regularization of neural networks using dropconnect. In *International conference on machine learning* (pp. 1058–1066). PMLR.
- Wang, B., Sun, Y., Xue, B., & Zhang, M. (2018). Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification. In *2018 IEEE congress on evolutionary computation* (pp. 1–8). IEEE.
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint [arXiv:1708.07747](https://arxiv.org/abs/1708.07747).